

Correct Solution: One 100% Dynamic Product Template

Executive conclusion

The live Shopify code confirms that the current Stride 8S page is not a single product component. It is a wrapper template that assembles multiple sections, while the main product-description section delegates most of its markup to a large snippet. The old implementation became SKU-specific because the wrapper, sections and snippet contain product handles, product IDs, membership IDs, prices, images, promotions and product copy.

The correct solution is **not** to copy the Stride 8S template and replace a few values. The correct solution is to preserve the reusable presentation layer and rebuild the data layer around the current Shopify product.

The final architecture should contain one public product template named `product.dynamic`, with `templates/product.liquid` optionally pointing to the same entry point for all products that use Shopify's default product template. That template must contain only neutral section references and universal layout logic. It must not contain a Stride name, SKU handle, product ID, membership variant ID, CDN product image, product price, promotion branch or product-specific URL.

What the live code contains

The live `product.cargo-stride-8s.liquid` wrapper assembles the following product-page sequence:

Order	Current live element	Correct dynamic treatment
1	Product description	Preserve the exact markup and classes, but read product data and profile metafields.
2	Membership upgrade	Keep reusable upgrade presentation; read membership offers from product data.
3	Product specifications	Render a reusable specification section from a product specification metaobject list.

4	Reviews	Keep the provider widget, but pass the current product ID, title, URL and price.
5	Experience	Keep the section layout; read product-specific media and copy from product content data.
6	Product experience CTA	Keep the CTA markup; read image, video, copy, price and features dynamically.
7	Feature slider	Render product feature items from a list of content records.
8	Full-width video	Read the product video, poster, heading and copy from product data.
9	Memberships	Use a reusable membership-program section with product-driven plan content.
10	Derisk section	Read returns, warranty and quality content from product profile data.
11	Financing CTA	Read finance label, rate and copy from product profile data.
12	More than a membership	Use configurable product or program content, not a Stride-specific section.
13	Featured collection	Read a collection reference from the product profile.
14	Community	Render product or brand community content from a reusable content record.
15	Safety	Read safety image and copy from product profile data.
16	Upgrade snippet	Keep only if the product profile enables the upgrade flow.
17	Membership comparison popup	Render comparison rows from reusable membership data, not fixed text in the snippet.

The navigation bar, mega menu, cart drawer and footer are global theme components and must remain outside the product template.

What must be preserved

The following are reusable presentation assets and should be retained from the approved reference implementation:

Presentation layer	Keep or rebuild
Product gallery layout	Keep the desktop and mobile markup, thumbnail behavior, image sizing, zoom behavior and CSS classes.
Product heading and price hierarchy	Keep the structure, but source title and prices from the product and product profile.
Membership card format	Keep the card geometry, radio controls, savings badges, free-delivery badge placement, benefit area and CTA styling.
Payment row	Keep the visual row and existing payment-provider assets.
Comparison modal layout	Keep the modal frame, table geometry and controls. Replace the fixed rows with dynamic comparison data.
Lower-page section order	Keep the order, but use neutral section names and product-driven data.
Existing CSS classes and reusable snippets	Keep where they are visual-only and do not contain SKU logic.
Product object usage	Keep native Shopify values such as <code>product.title</code> , <code>product.images</code> , <code>product.description</code> , selected variant and product URL.

What must be removed

The following dependencies make the current implementation SKU-specific and must not exist in the reusable template:

Dependency in current code	Replacement
<code>warrantyvariant = "39777944633457"</code>	Product profile warranty variant reference.
<code>var variantid = 41213675536497</code>	<code>product.selected_or_first_available_variant.id</code> .
<code>popupvariantid = 41213675536497</code>	The selected product variant from the product object.
<code>product.id == ... branches</code>	Product profile flags or metafields.
<code>product.handle != 'stair-climber-max'</code>	A product profile field such as <code>membership_enabled</code> .
Stride-specific section filenames	Neutral reusable section filenames.
Hardcoded membership card image URLs	Image references stored on membership-offer records.
Hardcoded membership prices and savings	Fields on membership-offer records.
Hardcoded membership benefits	List fields on membership-offer records.
Hardcoded promotion assets and gift IDs	Product promotion records with product-variant references.
Hardcoded sold-out contact copy and product URLs	Generic product-aware sold-out component using the current product object.
Fixed comparison rows	A list of comparison-row records.
Fixed shipping exception IDs	A product shipping policy field and a membership-offer <code>free_delivery</code> field.

Commenting out a hardcoded block is not enough. The block should be removed or rewritten so it cannot become active again by accident.

Recommended dynamic data model

1. Product profile metaobject

Create a single `product_template_profile` metaobject and attach it to each product using a product metafield such as `custom.template_profile` . This keeps complex product-page data out of the Liquid template.

Field	Type	Purpose
-------	------	---------

subtitle	Single-line text	Product subtitle or category line.
shipping_status	Single-line text or enum	ships_now , delayed or preorder .
shipping_date	Single-line text	Displayed when shipping is delayed.
price_label	Single-line text	Label such as From or Now only .
sale_price	Money	Optional product-specific sale price.
monthly_finance_price	Money	Monthly finance display.
finance_label	Single-line text	Finance label.
hero_banner_image	File reference	Product hero or banner image.
hero_video	URL or file reference	Optional product video.
hero_copy	Rich text	Product-specific hero copy.
specifications	List of specification metaobjects	Technical data rows.
experience_sections	List of content-section metaobjects	Feature, lifestyle and media content.
comparison_rows	List of comparison-row metaobjects	Product membership comparison rows.
related_collection	Collection reference	Product-specific related products.
safety_content	Rich text or content reference	Product safety and warranty copy.
membership_offers	List of membership-offer metaobjects	Plans displayed for this product.
membership_enabled	Boolean	Whether the purchase flow includes membership selection.
warranty_variant	Product variant reference	Optional warranty line item.
promotion_items	List of product-variant references	Optional product-specific extras.

2. Membership-offer metaobject

Each product can reference the same standard membership records or product-specific variants. The template never stores numeric variant IDs directly.

Field	Type	Purpose
plan_key	Single-line text	monthly , annual or twoyear .
program	Single-line text	premier or choice .
variant	Product variant reference	Membership item added to cart.
image	File reference	Card image.
title	Single-line text	Card title.
price_text	Single-line text or money	Display price.
savings_text	Single-line text	Savings badge text.
detail_text	Single-line text	Detail line under the price.
benefits	List of single-line text	Card benefits.
free_delivery	Boolean	Controls the free-delivery badge and banner.
extra_variants	List of product variant references	Heart-rate monitor, voucher, postage or other extras.
default_selected	Boolean	Default plan selection.

The delivery rule should be data-driven, but the template should also enforce the business rule that only annual and two-year offers may show free delivery. This prevents an incorrectly configured monthly offer from displaying the badge.

3. Specifications and content records

Use lists of metaobjects for repeating information instead of adding a new section or template per product.

A `specification_row` record should contain `label` , `value` , `column` and `sort_order` . A `product_content_section` record should contain `type` , `eyebrow` , `heading` , `copy` , `image` , `video` , `button_label` , `button_link` and `sort_order` . The dynamic renderer loops through the records and uses the existing reference markup for each content type.

Correct template architecture

The public template should become a small neutral wrapper:

Plain Text

```
{% assign profile = product.metafields.custom.template_profile.value %}
{% section 'product-header' %}
{% section 'product-banner' %}
{% section 'product-description' %}
{% section 'membership-upgrade' %}
{% section 'product-specifications' %}
{% section 'product-experience' %}
{% section 'feature-slider' %}
{% section 'full-width-video' %}
{% section 'memberships' %}
{% section 'derisk-section' %}
{% section 'financing-cta' %}
{% section 'more-than-membership' %}
{% section 'featured-collection' %}
{% section 'community' %}
{% section 'safety' %}
{% render 'membership-plans', profile: profile %}
{% render 'dynamic-cart-helper', product: product, profile: profile %}
```

The section names above are generic. The existing visual markup can be moved into these sections, but the sections should receive `profile` data and should not contain product-specific defaults. If a product does not have a profile or a content record, the section should either use native product data or render nothing, rather than showing another SKU's content.

The current `product.dynamic` name can be preserved. If every new SKU should receive the template automatically, `templates/product.liquid` should call the same neutral section sequence. Products assigned to old SKU-specific templates must then be migrated once to the default template.

Correct cart architecture

The cart helper should receive the selected product variant from the current product page and the selected membership offer from the product profile. It should add the selected product variant, membership variant, optional warranty variant and selected extra variants through one `/cart/add.js` request. No numeric product or membership variant IDs should be present in Liquid, JavaScript or section settings.

What is needed before implementation

The code and live theme are sufficient to build the neutral structure. The remaining business decisions are configuration choices, not additional code requirements:

Decision	Recommended default
Public template name	Keep <code>product.dynamic</code> .
Automatic use for new SKUs	Point <code>templates/product.liquid</code> to the neutral wrapper.
Product-specific data	Use one <code>custom.template_profile</code> metaobject reference per product.
Membership configuration	Use <code>membership_offer</code> metaobjects with product variant references.
Free delivery	Enforce only for annual and two-year offers.
Content without data	Hide the section or use a neutral universal fallback, never another SKU' s content.
Validation	Test at least two different products with the same template and compare cart line items.

The correct next implementation step is to create the metaobject definitions and neutral section wrappers, then migrate the exact reference markup into those wrappers one section at a time. Only after the schema and two-product test pass should the code be pushed to the theme.